MASTERY

FINAL MASTER PROMPT

AI-Powered Personal Operating System

Plan. Focus. Act. Grow.

Implementation blueprint for Claude Code in Visual Studio Code

Document Purpose

This document is the consolidated implementation prompt for Mastery. It merges the functional scope of the original Mastery and Compass specifications, removes duplicate requirements, and deliberately excludes the Compass UI design and color system. The retained visual direction is Mastery's design system.

MASTER PROMPT

PROJECT NAME

Mastery

PRODUCT CATEGORY

AI-Powered Personal Operating System and Personal Management Platform

CORE TAGLINE

Plan. Focus. Act. Grow.

TARGET DEVELOPMENT ENVIRONMENT

Claude Code inside Visual Studio Code

PRIMARY OBJECTIVE

Build Mastery as a secure, production-ready, multi-user personal operating system that helps each authenticated user plan, organize, execute, evaluate, and improve their life across three primary life pillars:

1. Spiritual

- Faith
- Character
- Spiritual disciplines
- Service
- Purpose

2. Personal

- Health
- Personal growth
- Finances
- Relationships
- Learning
- Habits
- Skills

3. Societal

- Work
- Career
- Business
- Leadership

- Community
- Social impact

Mastery must convert long-term vision into measurable daily execution.

The central workflow is:

Vision

```
-> Five-Year Plan
-> One-Year Plan
-> Quarterly Objectives
-> Monthly Objectives
-> Weekly Priorities
-> Daily Actions
-> Execution
-> Measurement
-> Reflection
-> Improvement
```

The product execution loop is:

PLAN

Define direction, plans, goals, projects, milestones, and priorities.

FOCUS

Allocate time, attention, energy, and calendar capacity.

ACT

Execute tasks, routines, habits, and commitments.

GROW

Reflect, learn, measure progress, develop skills, and improve future decisions.

1. ROLE OF THE AI CODING ASSISTANT

You are the senior full-stack architect and engineer responsible for building Mastery.

You must:

4. Inspect the repository before modifying files.
5. Read CLAUDE.md and all relevant files inside /docs.
6. Summarize the current repository state before implementation.
7. Identify existing dependencies and completed modules.
8. Propose a concise implementation plan.
9. Implement only the requested layer or sublayer.
10. Preserve all stable and completed functionality.
11. Avoid rebuilding existing modules unless a dependency requires a controlled refactor.

12. Reuse existing components, services, hooks, schemas, repositories, and utilities.
13. Keep page components thin.
14. Place business logic inside services, repositories, hooks, domain modules, server functions, or Cloud Functions.
15. Run type checking, linting, automated tests, and the production build.
16. Fix every failure before declaring the layer complete.
17. Update docs/BUILD_PROGRESS.md.
18. List every created, modified, moved, or deleted file.
19. Provide manual testing instructions.
20. Do not implement unrelated future features.
21. Do not introduce placeholder production data.
22. Do not expose secrets or privileged credentials.
23. Do not mark work complete when acceptance criteria have not passed.

2. PROJECT GOVERNANCE

Before implementing application functionality, create and maintain:

- CLAUDE.md
- docs/MASTER_SPEC.md
- docs/PRODUCT_REQUIREMENTS.md
- docs/ARCHITECTURE.md
- docs/DESIGN_SYSTEM.md
- docs/DATA_MODEL.md
- docs/SECURITY.md
- docs/AI_ARCHITECTURE.md
- docs/RECOVERY_PRIVACY.md
- docs/TESTING_STRATEGY.md
- docs/DEPLOYMENT.md
- docs/BUILD_PROGRESS.md
- docs/DECISIONS.md
- docs/CHANGELOG.md

CLAUDE.md must define:

- Repository rules
- Architecture rules
- Naming conventions
- Security requirements
- Testing requirements
- Layer-by-layer build procedure
- Prohibited implementation patterns
- Definition of done
- Required verification commands

BUILD_PROGRESS.md must record:

- Current layer
- Completed layers

- In-progress work
- Known defects
- Technical debt
- Deferred features
- Test status
- Build status
- Deployment status
- Next approved layer

3. PRIMARY TECHNOLOGY STACK

Use:

Frontend:

- Next.js App Router
- TypeScript with strict mode
- React
- Tailwind CSS
- Mastery's approved design system

Backend and Infrastructure:

- Firebase Authentication
- Cloud Firestore
- Firebase Storage
- Firebase Cloud Functions
- Firebase Cloud Messaging
- Firebase App Check
- Firebase Emulator Suite

Forms and Validation:

- React Hook Form
- Zod
- @hookform/resolvers

Data Visualization:

- Recharts

Internationalization:

- next-intl

Icons:

- Lucide React

Utilities:

- clsx

- tailwind-merge

Testing:

- Unit testing framework compatible with the repository
- React component testing
- Firebase Emulator rule tests
- Cloud Function tests
- Integration tests
- End-to-end tests
- Accessibility tests

Deployment:

- Vercel for the Next.js frontend
- Firebase for backend services
- GitHub for source control
- CI/CD for validation and preview deployments

Use stable dependency versions compatible with the existing repository. Do not upgrade packages without a documented technical reason.

4. PROJECT STRUCTURE

Use a modular, feature-oriented structure such as:

```
src/
  app/
  components/
    ui/
    layout/
    shared/
  features/
    auth/
    dashboard/
    vision/
    plans/
    goals/
    projects/
    milestones/
    tasks/
    habits/
    routines/
    calendar/
    focus/
    pomodoro/
    journal/
    learning/
    reading/
    skills/
    kpis/
    analytics/
    reports/
    ai-coach/
    recovery/
    notifications/
    settings/
  hooks/
  lib/
    firebase/
    validation/
```

```
      utils/
      security/
      analytics/
    providers/
    repositories/
    services/
    types/
    styles/

  functions/
src/
  ai/
      reports/
      notifications/
      recovery/
  scheduled/
      shared/

  docs/
  locales/
  public/
  tests/
```

Use clear domain boundaries. Do not create one oversized global service, hook, context, page, or component.

5. CORE ENGINEERING PRINCIPLES

24. Use strict TypeScript.
25. Do not use avoidable any types.
26. Validate all trust boundaries with Zod.
27. Keep UI, domain logic, data access, and infrastructure concerns separated.
28. Use reusable components.
29. Keep business logic outside page components.
30. Use authenticated user-scoped repositories.
31. Repositories must obtain the authenticated UID internally.
32. UI forms must never provide an arbitrary user ID for user-owned records.
33. Normalize Firebase errors.
34. Normalize Firestore timestamps.
35. Add loading, empty, success, and error states.
36. Add pagination to potentially large collections.
37. Avoid uncontrolled Firestore reads.
38. Use aggregation services for dashboard and report data.
39. Design mobile-first.
40. Support tablet and desktop layouts.
41. Use semantic HTML.
42. Support keyboard navigation.
43. Respect reduced-motion preferences.
44. Use accessible labels, focus states, and error messages.
45. Avoid hardcoded UI strings.
46. Avoid hardcoded user names and production statistics.
47. Use development fixtures only in controlled development environments.
48. Add audit fields to important records.
49. Document indexes and query requirements.
50. Prefer server-side enforcement over client-side assumptions.

6. MULTI-USER ARCHITECTURE

Mastery is a multi-user platform.

Each authenticated user must have:

- Their own profile
- Their own dashboard
- Their own plans
- Their own goals
- Their own projects
- Their own tasks
- Their own calendar records
- Their own habits
- Their own journal
- Their own KPIs
- Their own AI context
- Their own reports
- Their own Recovery Center
- Their own language and theme preferences

Users must never be able to access another user's private data.

Use owner-scoped Firestore paths or equivalent secure ownership controls.

Do not implement client-controlled administrator roles.

Any future administrative capability must be based on trusted server-issued claims and documented role-based authorization.

7. AUTHENTICATION

Implement:

- Email and password registration
- Email and password login
- Google authentication
- Logout
- Forgot-password flow
- Authentication persistence
- Protected routes
- Authenticated redirects
- Unauthenticated redirects
- Session loading state

- User profile creation
- User menu
- Authentication error normalization

Create:

- AuthProvider
- useAuth hook
- Authentication service
- Login page
- Registration page
- Forgot-password page
- Protected layout

On registration, create:

```
users/{uid}
```

User profile fields:

- id
- displayName
- email
- photoURL
- role
- language
- theme
- timezone
- accentColorPreference, if supported by the retained Mastery design system
- onboardingCompleted
- createdAt
- updatedAt

Default role:

```
user
```

Supported languages initially:

- English
- Dutch

Prepare the architecture for Spanish without requiring Spanish content in the first release.

Supported theme preferences:

- dark

- light
- system

Store preferences in the user profile and persist them across devices.

8. APPLICATION NAVIGATION

Create the following hierarchy.

DASHBOARD

PLAN

- Life Vision
- Five-Year Plans
- One-Year Plans
- Quarterly Plans
- Monthly Plans
- Weekly Plans
- Goals
- Projects
- Milestones
- Roadmaps

FOCUS

- Deep Work
- Pomodoro
- Priority Matrix
- Calendar
- Time Blocking
- Focus Sessions

ACT

- Tasks
- Habits
- Daily Routine
- Execution Tracker

GROW

- Journal
- Learning
- Reading
- Skills
- AI Coach

ANALYTICS

- KPIs
- Life Score
- Reports
- Trends

PRIVATE

- Recovery Center

SYSTEM

- Notifications
- Settings

Responsive behavior:

Desktop:

- Fixed left sidebar
- Sticky topbar
- Scrollable main content
- Optional contextual right panel

Tablet:

- Collapsible sidebar
- Compact topbar

Mobile:

- Navigation drawer
- Fixed bottom navigation for Dashboard, Plan, Focus, Act, and Grow
- Safe-area support
- Touch-friendly controls

Global shell functionality:

- Search shell
- Notification control
- User menu
- Command palette
- Breadcrumbs
- Active navigation states
- Loading states
- Error boundaries

9. CORE FIRESTORE DATA MODEL

Use typed models, Zod schemas, Firestore converters, repository interfaces, and Firebase repository implementations.

Suggested structure:

```
users/{uid}

users/{uid}/lifeVisions/{visionId}
users/{uid}/fiveYearPlans/{planId}
users/{uid}/yearPlans/{planId}
users/{uid}/quarterPlans/{planId}
users/{uid}/monthPlans/{planId}
users/{uid}/weekPlans/{planId}

users/{uid}/goals/{goalId}
users/{uid}/projects/{projectId}
users/{uid}/milestones/{milestoneId}
users/{uid}/roadmaps/{roadmapId}

users/{uid}/tasks/{taskId}
users/{uid}/events/{eventId}
users/{uid}/timeBlocks/{timeBlockId}
users/{uid}/focusSessions/{sessionId}
users/{uid}/pomodoroSessions/{sessionId}

users/{uid}/habits/{habitId}
users/{uid}/habitLogs/{logId}
users/{uid}/routines/{routineId}
users/{uid}/executionLogs/{logId}

users/{uid}/kpis/{kpiId}
users/{uid}/kpiEntries/{entryId}
users/{uid}/lifeScoreEntries/{entryId}

users/{uid}/journalEntries/{entryId}
users/{uid}/learningItems/{itemId}
users/{uid}/books/{bookId}
users/{uid}/skills/{skillId}

users/{uid}/weeklySummaries/{summaryId}
users/{uid}/reports/{reportId}
users/{uid}/notifications/{notificationId}
```

Recovery records must use separately protected subcollections documented in the Recovery Center section.

Common record fields:

- id
- userId, when needed internally
- createdAt
- updatedAt
- createdBy
- updatedBy
- status
- version
- archivedAt, when applicable

Use server timestamps where appropriate.

10. PLAN DOMAIN

The Plan domain must convert life direction into executable work.

10.1 Life Vision

Allow users to define:

- Personal mission
- Core values
- Life purpose
- Desired legacy
- Spiritual direction
- Personal direction
- Societal direction
- Long-term vision statements
- Future-self descriptions
- Non-negotiable principles

Each vision item must link to one or more life pillars.

10.2 Five-Year and One-Year Plans

Support:

- One-year plans
- Five-year plans
- Life-pillar assignments
- Objectives
- Desired outcomes
- Key measures
- Start and end dates
- Status
- Progress
- Review notes

10.3 Planning Cascade

Implement a controlled cascade:

Vision

```
-> Five-Year Plan
-> One-Year Plan
-> Quarter
-> Month
```

```
-> Week
-> Day
-> Task
```

The system must allow parent-child traceability.

A daily task should be traceable to its originating goal, project, plan, or life vision when such a relationship exists.

Do not automatically create excessive records without user confirmation.

Provide assisted planning recommendations while keeping the user in control.

10.4 Goals

Goals must support:

- Title
- Description
- Life pillar
- Parent plan
- Start date
- Target date
- Status
- Priority
- Progress
- Measurement type
- Target value
- Current value
- Unit
- Milestones
- Projects
- Tasks
- Habits
- KPIs
- Review frequency
- Notes
- Archive state

10.5 Projects

Projects must support:

- Goal linkage
- Life-pillar linkage
- Project status
- Owner
- Dates

- Priority
- Description
- Expected outcome
- Milestones
- Tasks
- Dependencies
- Risks
- Progress
- Review notes

10.6 Milestones

Milestones must support:

- Goal or project linkage
- Due date
- Completion state
- Progress
- Dependencies
- Evidence or notes

10.7 Roadmaps

Create timeline-oriented roadmaps for:

- Goals
- Projects
- Skill development
- Learning plans
- Personal transformation programs

11. FOCUS DOMAIN

11.1 Pomodoro

Implement:

- Configurable work duration
- Configurable short break
- Configurable long break
- Session cycles
- Pause
- Resume
- Cancel

- Completion
- Persistent session state
- Task linkage
- Goal linkage
- Project linkage
- Focus statistics
- Historical sessions

11.2 Deep Work

Support:

- Deep-work session creation
- Intended outcome
- Task or project linkage
- Start and end time
- Distraction log
- Energy level
- Focus quality
- Completion notes
- Session score

11.3 Calendar

Implement a functional internal calendar.

Support:

- Day view
- Week view
- Month view
- Event creation
- Event editing
- Event deletion
- Recurring events
- All-day events
- Reminders
- Goal linkage
- Project linkage
- Task linkage
- Time-block linkage
- Drag-and-drop where technically stable
- Timezone handling

Design the calendar service behind an adapter interface so that future Google Calendar, Outlook, or CalDAV synchronization can be added without rewriting the internal event model.

Do not implement external calendar synchronization until the internal calendar is stable.

11.4 Time Blocking

Allow users to allocate time to:

- Tasks
- Habits
- Goals
- Projects
- Deep work
- Learning
- Spiritual activities
- Recovery activities
- Personal commitments

Detect obvious scheduling conflicts and warn the user.

11.5 Priority Matrix

Support four quadrants:

- Urgent and important
- Important but not urgent
- Urgent but not important
- Neither urgent nor important

Allow tasks to be moved between quadrants.

12. ACT DOMAIN

12.1 Tasks

Tasks must support:

- Title
- Description
- Status
- Priority
- Due date
- Start date
- Life pillar
- Goal linkage

- Project linkage
- Milestone linkage
- Parent task
- Subtasks
- Recurrence
- Estimated duration
- Actual duration
- Energy requirement
- Context
- Tags
- Notes
- Completion timestamp
- Cancellation or postponement reason

12.2 Habits

Habits must support:

- Life-pillar linkage
- Goal linkage
- Frequency
- Schedule
- Target
- Unit
- Reminder
- Streak
- Longest streak
- Completion logs
- Missed logs
- Pause state
- Restart state
- Progress history

Include a habit streak tracker that is particularly suitable for spiritual disciplines, health habits, learning, and personal routines.

12.3 Daily Routine

Support:

- Morning routine
- Work or study routine
- Evening routine
- Custom routines
- Ordered steps
- Time estimates
- Habit linkage

- Completion tracking
- Reusable templates

12.4 Execution Tracker

The execution tracker must compare:

- Planned work
- Completed work
- Delayed work
- Cancelled work
- Rescheduled work
- Time estimated
- Time spent
- Focus quality
- Energy
- Reasons for non-completion

Use neutral, non-shaming language.

13. GROW DOMAIN

13.1 Journal

Support:

- Free-form entries
- Guided reflection
- Daily reflection
- Weekly reflection
- Gratitude
- Lessons learned
- Decision journal
- Goal linkage
- Life-pillar linkage
- Mood and energy metadata
- Tags
- Search
- Privacy controls

13.2 Learning

Support:

- Courses
- Study plans
- Lessons
- Notes
- Learning goals
- Completion tracking
- Study sessions
- Assessments
- Resources
- Goal and skill linkage

13.3 Reading

Support:

- Reading list
- Currently reading
- Completed books
- Reading progress
- Notes
- Highlights entered by the user
- Lessons
- Action items
- Goal linkage

Do not reproduce copyrighted book content that the user has not supplied.

13.4 Skills

Support:

- Skill inventory
- Current proficiency
- Target proficiency
- Skill categories
- Practice plans
- Evidence
- Learning resources
- Related goals
- Review dates
- Progress history

14. DASHBOARD

Create a dashboard aggregation service to prevent each widget from performing uncontrolled independent reads.

Dashboard data must be scoped to the authenticated user.

Dashboard widgets may include:

- Personalized greeting
- Current date
- Life Score
- Today's focus
- Today's priorities
- Today's schedule
- Habit streaks
- Energy check-in
- Focus hours
- Completed tasks
- Goal progress
- Project progress
- Weekly progress
- KPI overview
- AI Coach preview
- Habit preview
- Focus timer
- Quick notes
- Upcoming milestones
- Planning alignment
- Recovery check-in shortcut, without exposing sensitive details
- Notifications

Do not hardcode a user name.

Use real data when available and clear empty states when no data exists.

15. KPI, ANALYTICS, AND LIFE SCORE

Support:

- KPI definitions
- KPI entries
- Life-pillar linkage
- Goal linkage
- Manual entries
- System-calculated entries
- Time-series visualization
- Trends
- Comparisons
- Reporting periods
- Targets
- Thresholds

- Notes

Possible KPI categories:

Spiritual:

- Spiritual discipline consistency
- Service participation
- Reflection consistency

Personal:

- Health
- Exercise
- Sleep
- Financial progress
- Learning
- Habit consistency
- Relationship investment

Societal:

- Work performance
- Business progress
- Leadership
- Community contribution
- Social impact

Life Score:

Create a documented and configurable Life Score formula.

The score must:

- Use defined KPIs
- Show contributing factors
- Show weighting
- Avoid presenting itself as an objective judgment of human worth
- Allow the user to understand how the score was calculated
- Preserve historical entries
- Handle missing data fairly

Never generate an unexplained score.

16. AI COACH ARCHITECTURE

All AI requests must run through authenticated server-side Cloud Functions.

Never expose AI provider keys in client-side code.

Primary endpoint:

```
masteryCoachQuery
```

Possible supporting endpoints:

- generateWeeklySummary
- generatePlanningRecommendations
- generateGoalBreakdown
- generateReflectionQuestions
- analyzeExecutionPatterns
- recoveryCoachQuery

The general AI Coach must:

- Retrieve relevant user-authorized context
- Use plans, goals, tasks, habits, KPIs, and user-approved journal context
- Provide structured recommendations
- Explain which information influenced the recommendation
- Avoid inventing user records
- Clearly identify assumptions
- Allow users to accept, edit, or reject suggestions
- Avoid making irreversible changes automatically
- Keep recommendations aligned with user-defined values and priorities

AI infrastructure must include:

- Authentication
- Authorization
- Input validation
- Output validation
- Rate limiting
- Request logging
- Error normalization
- Timeouts
- Retry strategy where safe
- Cost controls
- Token limits
- Usage quotas
- Audit records
- Provider abstraction
- Safety controls
- Privacy boundaries

Use retrieval from Firestore only through trusted server code.

The AI Coach must not be given unrestricted access to all user data by default. Retrieve only context necessary for the current request.

17. WEEKLY AI SUMMARY

Create a scheduled weekly summary system.

The summary should evaluate the previous seven days and include:

- Accomplishments
- Completed goals or milestones
- Completed tasks
- Habit consistency
- Focus time
- Planning accuracy
- Delays
- Cancelled or postponed tasks
- Execution patterns
- KPI movement
- Lessons
- Suggested priorities for the next week

Store summaries under the authenticated user's records.

Notify the user when the summary is available.

The user must be able to review, archive, and delete stored summaries.

18. RECOVERY CENTER

The Recovery Center is a separate, private module intended to help users address self-identified behavioral patterns.

Examples may include:

- Procrastination
- Laziness or chronic avoidance
- Compulsive digital behavior
- Other user-defined recovery patterns

The module must not diagnose medical or psychological conditions.

It must not claim to replace a licensed healthcare professional.

18.1 Privacy Architecture

The Recovery Center must have stronger privacy controls than ordinary modules.

Requirements:

- Do not show sensitive recovery details on the standard dashboard.
- Do not expose recovery information in global search.
- Do not expose recovery information in ordinary notifications.
- Do not expose recovery information to other platform users.
- Require an additional privacy gate before opening the module.
- Support PIN protection initially.
- Architect for future biometric or WebAuthn support.
- Store recovery information in separately protected collections.
- Keep recovery AI conversations separate from general AI conversations.
- Never mix Recovery Coach context into the general planning AI.
- Never use recovery data for unrelated analytics or personalization.
- Provide data deletion controls.
- Document all recovery-data access paths.

18.2 Recovery Data Model

Suggested structure:

```
users/{uid}/recoveryProfiles/{profileId}
users/{uid}/recoveryGoals/{goalId}
users/{uid}/recoveryGoals/{goalId}/checkIns/{checkInId}
users/{uid}/recoveryGoals/{goalId}/relapses/{relapseId}
users/{uid}/recoveryGoals/{goalId}/copingActions/{actionId}
users/{uid}/recoveryAccountabilityPartners/{partnerId}
users/{uid}/recoveryCoachSessions/{sessionId}
```

A recovery goal may contain:

- User-defined behavior
- Start date
- Motivation
- Triggers
- Warning signs
- Coping strategies
- Support preferences
- Privacy settings
- Current status

18.3 Recovery Features

Support:

- Recovery goals
- Daily check-ins
- HALT check-in:
  - Hungry
  - Angry
  - Lonely
  - Tired
- Trigger identification
- Urge intensity
- Coping action selection
- Reflection
- Progress tracking
- Streak tracking
- Relapse or setback logging
- Restarting after a setback
- Coping toolkit
- Faith-based options selected by the user
- Evidence-informed behavioral techniques
- Recovery Coach
- Optional accountability partner

Use growth-oriented, neutral language.

Do not use shame-focused presentation.

Emphasize long-term progress rather than only current streak length.

18.4 Procrastination Integration

When a task is repeatedly cancelled, postponed, or left incomplete, allow the user to:

- Record the cause
- Identify a blocker
- Add a reflection
- Link the event to a recovery goal
- Ask the Recovery Coach for a non-judgmental next action

Do not automatically classify ordinary task delays as a behavioral problem.

18.5 Recovery Coach

Use a completely separate endpoint:

```
recoveryCoachQuery
```

The Recovery Coach must:

- Use a separate system instruction
- Be supportive and non-judgmental
- Focus on immediate safe next actions
- Use user-selected faith-based encouragement only when enabled
- Keep recovery conversations isolated from general AI sessions
- Avoid diagnosis
- Avoid coercive language
- Recommend professional or emergency assistance where appropriate

18.6 Accountability Partner

Accountability sharing must be opt-in.

The user controls:

- Which recovery goal is shared
- What information is shared
- How long access remains active
- Whether check-in reminders are sent
- Whether progress or setbacks are visible
- When access is revoked

The accountability partner must never receive direct Firestore access.

All partner access must pass through an authenticated and authorized Cloud Function.

Use explicit permissions such as:

- streak-only
- status-only
- check-in-completed
- selected-summary
- custom-limited-access

Never expose:

- Full journal content
- Private AI conversations
- Trigger details
- Sensitive notes
- Unapproved records

19. REPORTS AND PDF EXPORT

Support:

- Weekly reports
- Monthly reports
- Quarterly reports
- Annual reports
- Goal reports
- Habit reports
- Focus reports
- KPI reports
- Planning-versus-execution reports

PDF generation must be server-controlled where practical.

Reports must:

- Use authenticated user data
- Include selected reporting periods
- Explain metrics
- Handle missing data
- Avoid leaking hidden Recovery Center information
- Allow the user to choose included sections
- Use Mastery branding
- Produce a downloadable record
- Store export metadata where appropriate

Sensitive Recovery Center reports must require explicit separate user action and must never be included automatically.

20. NOTIFICATIONS

Support:

- In-app notifications
- Firebase Cloud Messaging
- Task reminders
- Event reminders
- Habit reminders
- Routine reminders
- Milestone reminders
- Weekly-summary notifications
- Planning-review reminders
- KPI-entry reminders

Notification requirements:

- User-controlled preferences
- Quiet hours
- Timezone awareness
- Category controls
- Read and unread states
- Safe deep links
- Permission handling
- Token lifecycle management
- Duplicate prevention

Recovery notifications must use privacy-safe wording and be separately configurable.

21. INTERNATIONALIZATION

Initial languages:

- English
- Dutch

Future language:

- Spanish

Requirements:

- Use next-intl.
- Store translations in locale files.
- Do not hardcode user-facing strings.
- Translate navigation, forms, validation messages, empty states, notifications, reports, and AI response wrappers.
- Detect browser language on first login when no preference exists.
- Allow users to change language from settings.
- Persist language preferences in Firestore.
- Restore the preference across sessions and devices.
- Use locale-aware dates, numbers, and times.
- Keep stored domain values language-neutral where possible.

22. THEME SUPPORT

Support:

- Dark
- Light

- System

Requirements:

- Use the retained Mastery design system.
- Do not implement the Compass color palette or Compass UI specification.
- Store the preference in the user profile.
- Respect the system preference when system mode is selected.
- Restore the preference after login.
- Avoid flashes of the incorrect theme during initial loading.
- Ensure accessibility and contrast in all supported themes.

23. PWA AND MOBILE READINESS

Build Mastery as an installable Progressive Web App.

Implement:

- Web app manifest
- Application name
- Short name
- Icons
- Theme metadata
- Mobile viewport configuration
- Installability
- Service worker
- Offline application shell
- Safe static-asset caching
- Update handling
- Touch support
- Mobile safe-area support
- Responsive layouts
- Keyboard support
- Minimum practical touch targets

Offline behavior must be explicit.

Do not imply that uncached cloud data is available offline.

Avoid unsafe caching of private or sensitive information.

The Recovery Center must receive additional scrutiny before any offline storage is enabled.

24. FIREBASE SECURITY

Security is mandatory, not optional.

Implement:

- Owner-only Firestore rules
- Owner-only Storage rules
- Authenticated Cloud Functions
- Input validation
- App Check
- Rate limits
- Secret management
- CSP
- Secure CORS configuration
- Environment separation
- Emulator-based rule testing
- AI endpoint authorization
- Recovery endpoint authorization
- Accountability-partner permission validation

Rules must ensure:

- Users can read and write only their own data.
- Users cannot assign themselves privileged roles.
- Users cannot access another user's Recovery Center.
- Accountability partners cannot directly query Recovery records.
- Sensitive access occurs only through approved Cloud Functions.
- Invalid record ownership is rejected.
- Required fields and valid values are enforced where practical.

Never rely only on hidden UI elements for authorization.

25. STORAGE SECURITY

All uploaded files must be:

- Scoped to the authenticated user
- Validated by file type
- Validated by size
- Stored in user-specific paths
- Protected by Storage rules
- Referenced through authorized records

Do not expose unrestricted public download URLs for private files.

Use short-lived or access-controlled retrieval patterns where necessary.

26. TESTING REQUIREMENTS

Implement:

- Unit tests
- Component tests
- Repository tests
- Zod schema tests
- Firestore rule tests
- Storage rule tests
- Cloud Function tests
- Integration tests
- End-to-end tests
- Accessibility tests
- Responsive tests

Critical journeys:

51. Registration
52. Login
53. Google authentication
54. Password recovery
55. Profile creation
56. Protected route access
57. User isolation
58. Goal creation
59. Goal-to-project linkage
60. Task creation and completion
61. Habit logging
62. Calendar event creation
63. Pomodoro persistence
64. KPI entry
65. Dashboard aggregation
66. Language persistence
67. Theme persistence
68. AI Coach authentication
69. Weekly summary generation
70. Recovery privacy gate
71. Recovery record isolation
72. Accountability permission enforcement
73. Report generation
74. PWA installation path

Every layer must pass:

- Type checking
- Linting
- Relevant automated tests
- Production build

Do not declare completion while failures remain.

27. DEPLOYMENT

Create separate environments:

- Development
- Test
- Staging
- Production

Use:

- Vercel for the frontend
- Firebase for Authentication, Firestore, Storage, Functions, Messaging, and App Check
- GitHub for version control
- CI/CD for validation and previews

CI/CD must run:

- Dependency installation
- Type checking
- Linting
- Tests
- Production build
- Rule tests where supported
- Security checks where supported

Do not deploy production secrets to preview environments.

Document:

- Environment variables
- Firebase projects
- Deployment commands
- Rollback process
- Database index deployment
- Rules deployment
- Function deployment
- App Check activation
- Domain configuration

28. PERFORMANCE AND OPTIMIZATION

Implement:

- Route-level code splitting
- Feature-level lazy loading
- Bundle analysis
- Firestore query review
- Firestore index review
- Pagination
- Data aggregation
- Controlled subscriptions
- Image optimization
- Caching where safe
- Cloud Function cold-start review
- AI token and cost review
- Core Web Vitals monitoring
- Accessibility review

Avoid:

- Loading all user data at application startup
- Permanent listeners where one-time reads are sufficient
- Duplicate dashboard queries
- Oversized client bundles
- Client-side AI provider SDKs containing privileged credentials
- Unbounded collection reads

29. BUILD ORDER

Build Mastery in the following order.

LAYER 0 - PROJECT CONSTITUTION

Create governance and documentation only.

LAYER 1 - PROJECT FOUNDATION

Initialize Next.js, TypeScript, Tailwind, linting, aliases, validation, environment handling, errors, loading, and repository structure.

LAYER 2 - MASTERY DESIGN SYSTEM

Implement only the retained Mastery design language and reusable components.

LAYER 3 - FIREBASE FOUNDATION

Initialize Firebase safely, add emulator support, converters, environment separation, Functions structure, and error normalization.

LAYER 4 - AUTHENTICATION AND USER ISOLATION

Implement authentication, profiles, protected routes, security rules, and rule tests.

LAYER 5 - APPLICATION SHELL AND NAVIGATION

Implement responsive protected layouts and placeholder module routes.

LAYER 6 - CORE DATA MODEL AND REPOSITORY LAYER

Implement schemas, types, converters, repositories, pagination, ownership, and audit fields.

LAYER 7 - DASHBOARD MVP

Implement operational aggregation and real user-scoped widgets.

LAYER 8 - PLAN DOMAIN

8A. Life Vision

8B. Five-Year and One-Year Plans

8C. Quarterly, Monthly, and Weekly Planning

8D. Goals

8E. Projects

8F. Milestones

8G. Roadmaps

8H. Planning Cascade

LAYER 9 - FOCUS DOMAIN

9A. Pomodoro

9B. Deep Work

9C. Calendar

9D. Time Blocking

9E. Priority Matrix

LAYER 10 - ACT DOMAIN

10A. Tasks

10B. Habits

10C. Daily Routine

10D. Execution Tracker

LAYER 11 - GROW DOMAIN

11A. Journal

11B. Learning

11C. Reading

11D. Skills

LAYER 12 - KPI, ANALYTICS, AND LIFE SCORE

LAYER 13 - GENERAL AI ARCHITECTURE

LAYER 14 - WEEKLY AI SUMMARY

LAYER 15 - RECOVERY CENTER

15A. Privacy Architecture

15B. Recovery Data Model

15C. Check-ins and Tracking

15D. Coping Toolkit

15E. Recovery Coach

15F. Accountability Partner

LAYER 16 - REPORTS AND PDF EXPORT

LAYER 17 - NOTIFICATIONS

LAYER 18 - INTERNATIONALIZATION AND THEME

LAYER 19 - PWA AND MOBILE READINESS

LAYER 20 - SECURITY HARDENING

LAYER 21 - COMPLETE TESTING PROGRAM

LAYER 22 - DEPLOYMENT AND CI/CD

LAYER 23 - PERFORMANCE, COST, AND ACCESSIBILITY OPTIMIZATION

Do not begin AI, Recovery Center, reports, push notifications, or external integrations before authentication, planning, tasks, habits, calendar, and KPI foundations are stable.

30. ACCEPTANCE CRITERIA FOR EVERY LAYER

A layer is complete only when:

75. The implementation matches the approved specification.
76. Existing functionality remains operational.
77. Type checking passes.
78. Linting passes.
79. Relevant tests pass.
80. The production build passes.
81. Security rules are tested where relevant.
82. Mobile behavior is verified.
83. Accessibility is checked.
84. Loading states exist.
85. Empty states exist.
86. Error states exist.
87. No secrets are exposed.
88. No unrelated future features were introduced.
89. BUILD_PROGRESS.md is updated.
90. Changed files are listed.
91. Manual test instructions are supplied.
92. Known limitations are documented honestly.

31. GIT STRATEGY

Branches:

- main
- develop
- feature/layer-[number]-[name]
- fix/[issue-name]
- security/[security-change]
- refactor/[controlled-refactor]

Examples:

- feature/layer-04-auth

- feature/layer-08-goals
- feature/layer-09-pomodoro
- feature/layer-15-recovery-gate
- security/recovery-owner-isolation

Commit examples:

- chore: initialize Mastery project foundation
- docs: add Mastery project constitution
- feat(auth): add Firebase email and Google authentication
- feat(goals): implement user-scoped goal management
- feat(calendar): add internal calendar event management
- feat(pomodoro): add persistent focus sessions
- feat(recovery): add private recovery check-ins
- security(recovery): enforce owner-only recovery records
- test(rules): verify cross-user access is denied
- perf(dashboard): consolidate user metric queries

32. REUSABLE CLAUDE CODE EXECUTION PROMPT

Implement Layer [NUMBER]: [LAYER NAME] for the Mastery project.

Before implementation:

93. Read CLAUDE.md.
94. Read all relevant documents in /docs.
95. Inspect the current repository.
96. Review docs/BUILD_PROGRESS.md.
97. Identify completed functionality that must be preserved.
98. Summarize the current implementation.
99. Identify dependencies and risks.
100. Provide a concise implementation plan.
101. Implement only the requested layer or sublayer.

Layer requirements:

[PASTE THE APPROVED LAYER REQUIREMENTS HERE]

Engineering rules:

- Use strict TypeScript.
- Validate trust boundaries with Zod.
- Keep business logic outside page components.
- Reuse the Mastery design system.
- Use authenticated user-scoped repositories.
- Derive the authenticated UID internally.

- Do not expose secrets.
- Do not add unrelated features.
- Do not rebuild completed modules unnecessarily.
- Preserve existing functionality.
- Add loading, error, empty, and success states.
- Add tests for critical logic.
- Use the Firebase Emulator Suite where relevant.
- Run typecheck, lint, tests, and production build.
- Fix every failure before completion.
- Update docs/BUILD_PROGRESS.md.
- List files created, changed, moved, or deleted.
- Provide manual testing instructions.
- State remaining limitations clearly.

Do not proceed to another layer after completing this layer.

33. FINAL PRODUCT STANDARD

Mastery must be:

- Secure
- Private
- Modular
- Maintainable
- Scalable
- Mobile-first
- Installable as a PWA
- Accessible
- Multi-user
- Internationalized
- AI-assisted
- User-controlled
- Transparent in its calculations
- Careful with sensitive information
- Suitable for continuous development

The final product must not function merely as a collection of disconnected productivity tools.

Every major module must connect back to the central Mastery operating system:

Plan

```
-> Focus
-> Act
-> Grow
```

Long-term direction must connect to daily execution.

Daily execution must generate measurable progress.

Measured progress must support reflection.

Reflection must improve the next planning cycle.